Davante

13

PROGRAMACIÓN

Excepciones y manejo básico de ficheros, directorios y archivos

ÍNDICE

/ 1. Introducción y contextualización práctica 4

/ 2. Excepciones 5

/ 3. Control de excepciones 6

/ 4. Caso práctico 1: “Manejo de muchas excepciones” 7

/ 5. Lanzar excepciones 7

/ 6. Propagar excepciones 8

/ 7. Creación de excepciones propias 9

/ 8. Documentación de excepciones 10

/ 9. Clases de acceso a ficheros 11

/ 10. Lectura de ficheros de texto 12

/ 11. Escritura de ficheros de texto 13

/ 12. Caso práctico 2: “Escritura de ficheros” 14

© MEDAC ISBN: 978-84-19885-69-2
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

ÍNDICE

/ 13. Directorios 15

13.1. Crear directorios 15

13.2. Listar archivos de un directorio 15

13.3. Eliminar un directorio 15

/ 14. Creación y eliminación de archivos 16

/ 15. Resumen y resolución del caso práctico de la unidad 17

/ 16. Bibliografía 18

OBJETIVOS

Conocer y gestionar excepciones.

Propagar excepciones.

Crear nuevas excepciones.

Leer información de ficheros de texto.

Almacenar información en ficheros de texto.

Crear y eliminar archivos y directorios.

/ 1. Introducción y contextualización práctica

En este tema se hablará de los errores en los programas y de qué son las excepciones. Veremos cómo podemos tratar estos errores o excepciones para que el programa siga funcionando o no cierre de manera abrupta.

También aprenderemos a crear nuestras propias excepciones para manejar errores personalizados y a propagarlas correctamente.

Vamos a ver, asimismo, cómo podemos escribir y leer información de ficheros de texto, además de la manera de tratarlos de forma correcta.

Por último, nos detendremos en el tratamiento de archivos y directorios.

A continuación, vamos a plantear un caso práctico a través del cual podremos aproximarnos de forma práctica a la teoría de este tema.



Fig.1. Safecode.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema. Encontrarás su resolución en el apartado 'Resumen y resolución del caso práctico'.

Audio intro. "Errores de código"

https://on.soundcloud.com/ExbL4NvLtyYrWKeS7

/ 2. Excepciones

A lo largo de todas las unidades anteriores nos hemos encontrado con que nuestros programas han provocado errores tras hechos tan simples como olvidarnos de terminar una instrucción con un punto y coma, cerrar unas comillas, etc. Fallos de los que el propio compilador nos avisa, de hecho no nos deja compilar hasta que los arreglemos.

Existen otro tipo de errores, concretamente **los errores en tiempo de ejecución**. Estos son los que se dan durante la ejecución del programa, es decir, el compilador no nos avisa de que vamos a tenerlos, sino que los descubriremos por nosotros mismos.

Estos son los más 'peligrosos'. Cuando se comete un error en tiempo de ejecución, el programa termina de forma abrupta, se cierra sin dar la oportunidad de guardar las acciones que se hayan realizado, fuerza al usuario a volver a ejecutarlo y a rehacerlo todo de nuevo.

El **manejo de las excepciones** va a hacer que los programadores tengan que crear un bloque exclusivo para el tratamiento de error, provocando que el programa, cuando lo detecte, vaya directo a ejecutar ese bloque, consiguiendo que el programa no se cierre de forma abrupta y, por tanto, permitiendo al usuario volver a realizar la acción que ha provocado el error de una forma segura.

Con el tratamiento de excepciones vamos a conseguir que los usuarios no noten que el programa ha fallado, aunque lo más recomendable es avisarlos de alguna forma de que ha ocurrido un fallo, pero el programa se ha recuperado de forma exitosa de ello.

Enlaces de interés...

La lista de excepciones internas que nos proporciona Java la podemos consultar en el siguiente enlace: https://docs.oracle.com/javase/7/docs/api/java/lang/Exception.html

Algunos **ejemplos** que pueden provocar una excepción son: no coincide una lectura de tipos de datos (InputMismatchException), una división entre cero (ArithmeticException), el intentar acceder a una posición no válida de un *array* (IndexOfBoundsException), un desbordamiento aritmético, un acceso fallido a disco o los errores propios del tema relacionados con ficheros como:

- El fichero al que queremos acceder no existe (FileNotFoundException).
- El fichero al que queremos acceder está corrompido.
- No se tienen permisos de lectura sobre el fichero (IOException).
- No se tienen permisos de escritura sobre el fichero.

Las excepciones en Java heredan todas de la clase Exception.

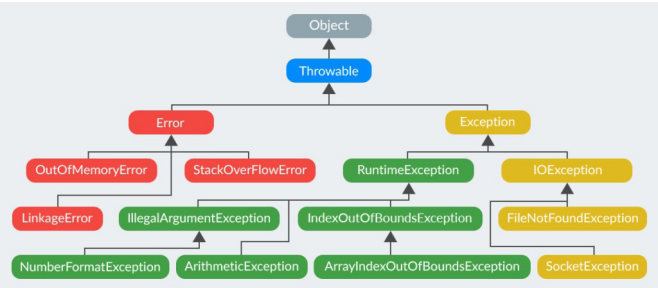


Fig.2. Excepciones de Java.

/ 3. Control de excepciones

En Java, el control de excepciones lo vamos a realizar mediante los **bloques try-catch**.

Estos bloques nos van a permitir **'aislar' un fragmento de código que sea susceptible de producir un error** e indicar el código que queremos que se ejecute en caso de que ese error se produzca.

En el **bloque try** vamos a escribir todo el código que sea susceptible de producir uno o varios errores. Este código irá en el cuerpo del try, que estará definido mediante llaves ({ }).

En el **bloque catch** vamos a escribir todo el código que queremos que se ejecute para tratar el error que se capture.

Hay **excepciones** que nos las detectará automáticamente el propio compilador, que nos avisará de que tenemos que poner un bloque try-catch, pero, en otras ocasiones, no lo hará y lo descubriremos nosotros mismos cuando el programa falle.

Cuando se produce una excepción, NetBeans nos muestra dónde ha ocurrido y el tipo de excepción que se ha generado, pudiendo así, de una forma sencilla, encapsular ese fragmento de código en un bloque try-catch y controlar esa excepción.

Un try puede tener uno o varios catch, pero un catch no puede estar sin un try. Cuando un try tiene varios catch, podemos agruparlos en un multicatch, tratando todas esas excepciones de igual forma. Para separar las diferentes excepciones en un multicatch, utilizaremos el símbolo '|' entre los nombres de las mismas.

En el catch podemos ver que tenemos un objeto de una clase que representa una excepción. Mediante este objeto, podremos saber cuál ha sido el motivo del error.

Una vez que se produzca un error, se pasará el control directamente al catch, provocando que todo el código que se encuentre por debajo de la línea que ha provocado el error, no se ejecute.

En la siguiente figura podemos ver un ejemplo de bloque try-catch:

```
public class Main {
    public static void main(String[] args) {
        try {
            int numerador = 10;
            int denominador = 0;
            int resultado = numerador / denominador;
            System.out.println("El resultado de la división es: " + resultado);
        } catch (ArithmeticException e) {
            System.out.println("Error: División entre cero.");
        }
    }
}
```

Fig.3. Excepción división 0.

¡! Sabías que...

El bloque finally en una estructura try-catch se utiliza para asegurar que un conjunto de instrucciones se ejecute sin importar si se lanza o no una excepción. Esto es especialmente útil para liberar recursos como archivos, conexiones a bases de datos o flujos de entrada/salida.

/ 4. Caso práctico 1: "Manejo de muchas excepciones"

Planteamiento: En ocasiones es necesario manejar múltiples excepciones relacionadas que permiten hacer uso del multicatch para reducir la duplicidad de código.

El uso del multicatch hace posible capturar varias excepciones en un solo bloque catch, mejorando la legibilidad y el mantenimiento del código. Esto es especialmente útil cuando diferentes excepciones requieren la misma lógica de manejo, haciendo el código más limpio y eficiente.

Nudo: Se está realizando un programa donde es necesario tratar varias excepciones, en concreto 6 excepciones diferentes que pueden ocurrir en el fragmento de código que se está desarrollando.

Con tantas excepciones se va a tener más código en bloques catch que el propio código con la lógica del programa. ¿Cómo crees que se puede elaborar el código de forma más ordenada?

Desenlace: Cuando hay una única excepción a tratar, o incluso dos, podemos poner uno o dos bloques catch en el try, pero cuando ya son muchas, el desarrollo del código se puede tornar un poco lioso a la hora de gestionar las excepciones.

En estas situaciones, lo más habitual y recomendable es el uso del multicatch, ya que gracias a él se pueden agrupar en un único catch varias excepciones, consiguiendo así reducir de forma considerable la cantidad de líneas de código del programa.

Hay que tener cuidado, ya que, como es bien sabido, no es bueno generalizar, sin embargo, al tener varias excepciones, se podrá usar el multicatch mientras se quiera dar el mismo tratamiento a todas.

Cuando se desee hacer alguna distinción, se deberá usar más de un catch, aunque siempre se podrán emplear varios multicatch para mitigar, en lo posible, esto, como se muestra en la imagen:

```
public static void main(String[] args) {
    double a = 4, b = 5;

    try {
        double resultado = a / b;
        System.out.println("Resultado: " + resultado);
    } catch (NullPointerException | ArithmeticException ex) {
        System.out.println("ERROR:" + ex.getMessage());
    }
}
```

Fig.4. Uso del multicatch.

/ 5. Lanzar excepciones

Ya hemos visto cómo podemos tratar un fragmento de código que provoca excepciones mediante un bloque try-catch.

Puede ocurrir que, en algún punto, necesitemos **lanzar una excepción por nosotros mismos**, ya sea porque estemos ante una excepción personalizada, o por especificaciones del proyecto que estemos desarrollando.

Imaginemos, por ejemplo, que estamos realizando una aplicación donde estamos haciendo un registro de personas. Estas, entre muchos atributos, tendrán una edad, y un requisito para poder llevar a cabo el registro será que esas personas sean mayores de edad, es decir, que la edad sea mayor o igual que 18. Podemos gestionar nuestro código para que, cuando esto ocurra, se lance una excepción de forma automática.

Para lanzar excepciones podemos usar la **instrucción throw**, a la cual deberemos indicar la excepción que queremos que lance, utilizando su constructor con parámetros e indicando el mensaje que queremos que aparezca cuando se consulte el motivo del error.

La instrucción throw solo puede lanzar una excepción a la vez. Cuando lancemos una excepción, todo el código que haya por debajo no se ejecutará.

En un programa, podremos lanzar todas las excepciones que sean necesarias.

Es importante recalcar que, cuando se ha producido la excepción y se ha cedido el control al bloque catch, al terminar de ejecutarse este, se seguirá con la ejecución normal del programa.

Un ejemplo de lanzamiento de excepciones lo podemos ver en la siguiente figura:

```
public static void main(String[] args) {
    Scanner teclado_int = new Scanner(System.in);
    int edad;

    try
    {
        System.out.println("Introduce la edad de la persona");
        edad = teclado_int.nextInt();
        if( edad < 18 )
        {
            throw new Exception("La persona ha de ser mayor de edad");
        }
    }
    catch(Exception error)
    {
        // Mostramos la causa del error
        System.out.println("Error: " + error.toString());
    }
}
```

Fig.5. Lanzando una excepción.

/ 6. Propagar excepciones

Una vez que hemos aprendido cómo podemos controlar las excepciones y lanzarlas según nuestras necesidades, vamos a ver cómo podemos **propagar una excepción por nuestro programa** de forma correcta, para que pueda ser tratada en el programa principal.

En la propagación de excepciones entran en juego los **métodos**. Ya sabemos qué es un método y cómo funciona, pero ¿qué ocurre cuando en un método se produce un error?

La propagación de excepciones en Java se refiere al proceso de pasar una excepción de un método a otro a lo largo de la pila de llamadas de método, de manera que, al producirse una excepción en un método, podemos propagarla para que el bloque de código donde este fue invocado se encargue de ella.

Para propagar una excepción en un método usamos la **instrucción throws**, a la cual podremos indicar cuál o cuáles excepciones lanza el método. Para indicar que queremos propagar más de una excepción en un método, pondremos una lista de ellas separadas por comas.

La sintaxis de un método que propaga excepciones es la siguiente:

```
[public / private] [tipo devuelto / void] nombreMetodo ([parámetros]) throws Excepcion1, Excepcion2
```

Una vez que un método tiene la sentencia throws para indicar que puede propagar una o más excepciones, cuando lo invoquemos desde cualquier parte de nuestro programa, el compilador nos mostrará un error que nos dirá que el método llamado provoca una excepción, teniendo que usar un bloque try-catch para su uso.

Puede ocurrir que haya excepciones que ya contengan implícitamente a otras, es decir, que, en el abanico de posibilidades de error que recoge una excepción, estén cubiertas todas las posibilidades que abarca otra, como se muestra en la siguiente imagen:

```
public class Main {
    public static void main(String[] args) {
        try {
            metodoA();
        } catch (Exception e) {
            System.out.println("Excepción capturada en el método main: " + e.getMessage());
        }
    }

    public static void metodoA() throws Exception {
        metodoB();
    }

    public static void metodoB() throws Exception {
        metodoC();
    }

    public static void metodoC() throws Exception {
        // Simulando una excepción
        throw new Exception("Ocurrió un error en el método C");
    }
}
```

Fig.6. Propagación de excepción.

En estos casos, tendremos que eliminar la excepción contenida, ya que con la excepción que la contiene es más que suficiente para el tratamiento de los posibles errores.



Audio 1. "Propagando excepciones de forma correcta"

https://bit.ly/3lOU2B4



/ 7. Creación de excepciones propias

En Java, al igual que en muchos otros lenguajes de programación, además de las excepciones definidas en el propio lenguaje, vamos a poder crear **nuestras propias excepciones para tratar casos específicos**.

Ya hemos visto anteriormente que todas las excepciones heredan de la clase Exception. Esto quiere decir que si en un bloque try-catch atrapamos la excepción Exception, atraparemos todas y cada una de las excepciones que se produzcan en el bloque try.

Cuando creemos **una excepción**, a la hora de nombrarla, esta **deberá terminar en Exception**, no como forma obligatoria, sino como parte de un estándar, **para que** al verla **se sepa** inmediatamente **que representa un posible error en nuestro código**.

Por ejemplo, si queremos crear una excepción para que se lance cuando un DNI introducido no sea válido, podríamos llamarla DNIException.

Una excepción va a ser una clase nueva que tendrá que heredar de la clase Exception, pero solo con esto no basta, deberemos implementar el constructor con parámetros de esta nueva clase, al que le llegará como parámetro un objeto de tipo String, que será el mensaje que propagará nuestra nueva excepción.

Estas nuevas excepciones podrán ser utilizadas en los bloques catch, pudiendo lanzarlas cuando lo necesitemos mediante la instrucción throw, y propagarlas en los métodos mediante la instrucción throws. Al fin y al cabo, lo que tenemos es una excepción totalmente funcional.

En un programa, podremos crear tantas excepciones como necesitemos.

A la hora de crear nuestras propias excepciones (imagen) es importante asegurarse bien de que no estén ya cubiertas por las que Java nos ofrece de forma interna, para así poder evitar la duplicación innecesaria de código.

```java
// Definición de una excepción personalizada
class MiExcepcion extends Exception {
    public MiExcepcion(String mensaje) {
        super(mensaje);
    }
}

public class Main {
    public static void main(String[] args) {
        try {
            // Lanzar la excepción personalizada
            throw new MiExcepcion("¡Esto es una excepción personalizada!");
        } catch (MiExcepcion e) {
            // Capturar y manejar la excepción personalizada
            System.out.println("¡Se ha producido una excepción personalizada!");
            System.out.println("Mensaje de la excepción: " + e.getMessage());
        }
    }
}
```

Fig.7. Creación de una excepción.

Vídeo 1. "Creación y uso de una excepción"

https://bit.ly/31WS0XD

/ 8. Documentación de excepciones

Ya vimos en unidades anteriores la necesidad de documentar los programas para que sea mucho más fácil la tarea de leerlos para entender qué hacen. Las **excepciones** no son menos, así que también habrá que **documentarlas**, ya que así será más fácil **saber qué tipo de errores tratan**.

Cuando creemos una **excepción propia**, la documentación será la misma que se utiliza al documentar una clase, ya que, al fin y al cabo, lo que creamos es una **clase**.

Cuando tenemos un método que lanza una o varias excepciones, tendremos que agregar la **etiqueta @throws** por cada una de las excepciones que lancemos.

La descripción que debemos poner en cada @throws es el caso en el que se lanzará esa excepción, por ejemplo, imaginemos que tenemos un método que lanza una excepción cuando se divide por cero, deberíamos indicar que esa excepción se va a lanzar al producirse una división entre cero, como muestra la imagen:

```java
/**
 * Divide dos números enteros.
 *
 * @param numerador el numerador de la división
 * @param denominador el denominador de la división
 * @return el resultado de la división
 * @throws ArithmeticException si el denominador es cero
 */
public int dividir(int numerador, int denominador) throws ArithmeticException {
    if (denominador == 0) {
        throw new ArithmeticException("División por cero");
    }
    return numerador / denominador;
}
```

Fig.8. Documentación de una excepción.

Algunas pautas para documentar las excepciones serían las siguientes:

- Utilizaremos Javadoc o Doxygen para documentar las excepciones lanzadas por nuestros métodos. Incluiremos etiquetas @throws en la documentación del método para indicar qué excepciones pueden ser lanzadas y en qué circunstancias.
- Proporcionaremos una descripción detallada de las excepciones lanzadas, incluyendo cuándo pueden ocurrir y qué significan. Esto ayuda a los desarrolladores a entender las condiciones en las que deben manejar esas excepciones.
- Si nuestro método lanza excepciones verificadas (aquellas que heredan de Exception pero no de RuntimeException), los consumidores del método estarán obligados a manejarlas. Documentaremos estas excepciones adecuadamente para que los usuarios sepan cómo manejarlas.
- Para excepciones no verificadas (aquellas que heredan de RuntimeException), la documentación es igualmente importante, pero los consumidores del método no están obligados a manejarlas. Sin embargo, aún debemos proporcionar información sobre estas excepciones, para que los desarrolladores sepan cómo podrían ocurrir y cómo manejarlas si es necesario.
- Proporcionaremos ejemplos de uso que incluyan cómo manejar las excepciones que pueden ser lanzadas por nuestros métodos.

/ 9. Clases de acceso a ficheros

Ya hemos visto que tenemos varios tipos de flujos a ficheros. El lenguaje de programación Java nos va a permitir trabajar con todos y cada uno de los tipos de flujos que hemos visto, y tiene ya integradas clases para tal propósito.

Todas las clases que necesitaremos estarán dentro del **paquete java.io**, por lo que tendremos que importarlas en el momento en el que las usemos.

Si recordamos lo visto en unidades anteriores, no es recomendable importar un paquete entero, ya que importaremos clases que no necesitamos, sino que lo que tendremos que hacer es **ir importando una a una las clases** que vayamos a usar.

Las **clases** que vamos a necesitar para poder **trabajar con flujos** son las siguientes:

- **File**: Para abrir un fichero.
- **FileReader**: Para generar un flujo de lectura carácter a carácter a un fichero de texto.
- **BufferedReader**: Para generar un flujo de lectura línea a línea a un fichero de texto.
- **FileWriter**: Para abrir un flujo de escritura carácter a carácter a un fichero de texto.
- **PrintWriter**: Para abrir un flujo de escritura línea a línea a un fichero de texto.
- **FileOutputStream**: Para generar un flujo de escritura carácter a carácter a un fichero binario.
- **BufferedOutputStream**: Para generar un flujo de escritura línea a línea a un fichero binario.

- **FileInputStream**: Para generar un flujo de lectura carácter a carácter a un fichero binario.
- **BufferedInputStream**: Para generar un flujo de lectura línea a línea a un fichero binario.
- **RandomAccessFile**: Para generar un flujo, tanto de lectura como de escritura, a un fichero de acceso aleatorio.

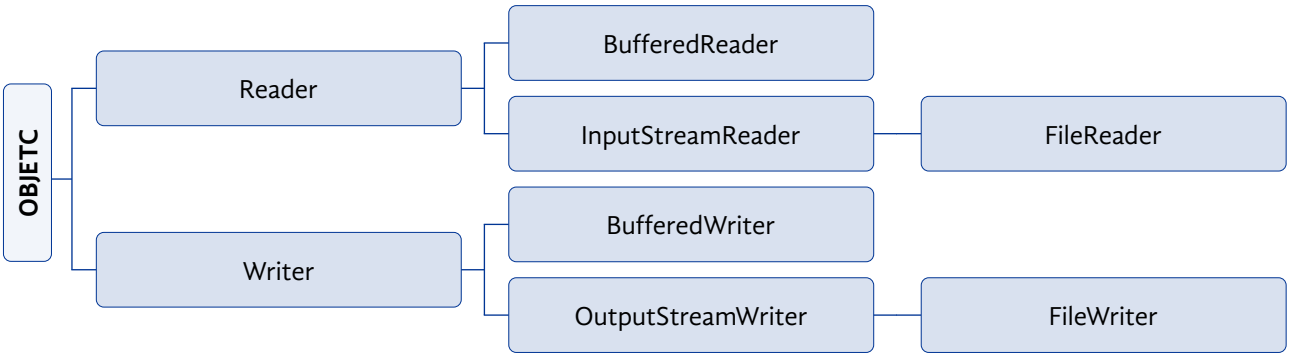


Fig.9. Clases de acceso a ficheros .

Recuerda...

IOException es una excepción común que puede ocurrir al trabajar con ficheros en Java. Esta excepción se lanza cuando ocurre un error durante una operación de entrada/salida (E/S) en el sistema de archivos.

/ 10. Lectura de ficheros de texto

Para **leer los datos** de un fichero de texto, ya adelantamos algunas **clases** anteriormente, pero concretamente necesitamos las siguientes:

- **File**: Nos permitirá abrir un fichero.
- **FileReader**: Nos servirá para poder generar un flujo de lectura carácter a carácter a un fichero de texto.
- **BufferedReader**: Podremos generar un flujo de lectura línea a línea a un fichero de texto.

Los **pasos a seguir para leer datos de un fichero de texto** son los siguientes:

1. Creamos un objeto de tipo File.
2. Creamos un objeto de tipo FileReader a partir del objeto de tipo File. Con esto podremos leer datos carácter a carácter.
3. Creamos un objeto de tipo BufferedReader a partir del objeto de tipo FileReader. Con esto podremos leer líneas completas del fichero.
4. Leemos los datos que necesitemos. Para leer una línea utilizaremos el método readLine de la clase BufferedReader, el cual nos devolverá la línea de texto correspondiente, devolviendo null si se ha llegado al final del fichero. Si queremos leer todo el contenido, lo haremos mediante un bucle while.
5. Cerramos el fichero. Para ello utilizamos el método close de las clases BufferedReader y FileReader. Es importante cerrar primero el objeto de BufferedReader y después el de FileReader.

En caso de que el fichero que queramos leer no exista, se producirá un error. Los datos se leerán en el mismo orden en el que se escribieron.

```
try {
    // Apertura del fichero y creación de BufferedReader para poder
    // hacer una lectura cómoda (dispone del método readLine()).
    archivo = new File("archivo.txt");
    fr = new FileReader(archivo);
    br = new BufferedReader(fr);

    // Lectura del fichero
    String linea;
    while ((linea = br.readLine()) != null) {
        System.out.println(linea);
    }
} catch (IOException e) {
    System.out.println("Error leyendo el fichero: " + e.toString());
} finally {
    try {
        br.close();
        fr.close();
    } catch (IOException e2) {
        System.out.println("Error cerrando el fichero: " + e2.toString());
    }
}
```

Fig.10. Lectura de fichero archivo.txt.

Este código imprimirá el contenido del archivo línea por línea en la consola. Si necesitamos procesar el contenido de otra manera, podemos ajustar la lógica dentro del bucle while.



Audio 2. "Leyendo datos"
https://bit.ly/353FqrN



/ 11. Escritura de ficheros de texto

Los **ficheros de texto** son aquellos cuya **codificación** es **en caracteres**, esto quiere decir que podremos editarlos con cualquier editor de texto, como bloc de notas.

Para escribir en un fichero de texto, necesitamos las siguientes clases:

- **FileWriter**: Podremos abrir un flujo de escritura carácter a carácter a un fichero de texto.
- **PrintWriter**: Nos permitirá abrir un flujo de escritura línea a línea a un fichero de texto.

Los **pasos a seguir para escribir en un fichero de texto** son los siguientes:

1. Crear un objeto de tipo FileWriter a partir de la ruta del fichero. Con esto crearemos el flujo de escritura hacia el fichero, pero solo podremos escribir carácter a carácter.
2. Crear un objeto de tipo PrintWriter a partir del objeto FileWriter para poder escribir cadenas de texto completas en una operación.
3. Escribir las cadenas que necesitemos a partir del método println de la clase PrintWriter. Este método funciona exactamente igual que cuando lo usamos para mostrar información en pantalla, escribirá una línea en el fichero e, inmediatamente después, un salto de línea.
4. Cerrar el fichero. Para ello utilizamos el método close de las clases FileWriter y PrintWriter. Es importante cerrar primero el objeto de PrintWriter y después el de FileWriter.

Una vez que hayamos ejecutado el código, tendremos el fichero creado con el texto introducido.

Si el fichero en el que queremos escribir no existe, se creará, y si existe, se sobrescribirá.

Para que no se sobrescriba el texto tendremos que pasarle un parámetro extra al constructor del objeto de FileWriter añadiendo un parámetro como true, que indicará que se escriba, al final del fichero, el texto nuevo.

```
try {
    fichero = new FileWriter("prueba.txt");
    pw = new PrintWriter(fichero);

    for (int i = 0; i < 10; i++) {
        pw.println("Línea " + i);
    }
} catch (IOException e) {
    System.out.println("Error leyendo el fichero: " + e.toString());
} finally {
    try {
        pw.close();
        fichero.close();
    } catch (IOException e2) {
        System.out.println("Error cerrando el fichero: " + e2.toString());
    }
}
```

Fig.11. Escritura de fichero prueba.txt.

/ 12. Caso práctico 2: "Escritura de ficheros"

Planteamiento: A la hora de escribir texto, el FileWriter de Java permite hacerlo en ficheros de manera eficiente. Para ello, es necesario crear una instancia de FileWriter y utilizar el método write para escribir el contenido deseado.

Nudo: Se pide la implementación para la escritura de texto en ficheros. Se ha de tener en cuenta que posiblemente se pida guardar información en el fichero en distintas partes del código. ¿Cómo crees que puede resolverse este ejercicio? ¿Tiene sentido realizar una función para una operación tan básica como esta?

Desenlace: Al estar programando cualquier función que auguremos que nos puede simplificar las tareas, por sencillas que estas puedan ser, merece la pena hacerla, o, por lo menos, intentar hacerla, ya que si lo conseguimos de una manera relativamente fácil, ahorrará mucho tiempo de desarrollo a futuro.

En este caso, se podrá hacer que esta función ya controle algunos de los posibles errores que puede ocasionar el uso de ficheros, concretamente, la escritura en un fichero. Dicho esto, esta función deberá controlar y lanzar las excepciones pertinentes, aunque, si no tenemos claro cuáles son estas (ya que aún no hemos hablado del tema), podemos hacer que controle la excepción general, chequeando así los errores producidos de una manera sencilla mediante la solución automática de los *warnings* que nos proponga el IDE.

Un ejemplo de la función solicitada podría ser la mostrada en la siguiente imagen:

```
public static void escribirFichero(String content) throws IOException{
    // Nombre del fichero en el que queremos escribir
    String fileName = "ficheroEscritura.txt";

    // Crear una instancia de FileWriter
    try (FileWriter writer = new FileWriter(fileName)) {
        // Escribir el contenido en el fichero
        writer.write(content);
        // Información adicional
        writer.write("\nAñadimos otra línea de texto.");
        System.out.println("Escritura completada.");
    } catch (IOException e) {
        // Manejo de excepciones en caso de error de escritura
        System.err.println("Ocurrió un error al escribir en el fichero: " + e.getMessage());
    }
}
```

Fig.12. Escritura de texto en fichero.

/ 13. Directorios

Al igual que podemos crear y eliminar ficheros, el lenguaje de programación **Java también permite realizar varias operaciones con directorios** utilizando la clase File o las clases más modernas introducidas en Java 7, como Path y Files del paquete java.nio.file.

13.1. Crear directorios

Podemos crear un directorio utilizando el método mkdir() o mkdirs() de la clase File. La diferencia entre ellos es que mkdirs() también creará los directorios padres si no existen.

```java
import java.io.File;

public class Main {
    public static void main(String[] args) {
        File directorio = new File("mi_directorio");
        if (!directorio.exists()) {
            if (directorio.mkdir()) {
                System.out.println("Directorio creado correctamente.");
            } else {
                System.out.println("Error al crear el directorio.");
            }
        }
    }
}
```

Fig.13. Creación de un directorio.

13.2. Listar archivos de un directorio

Puedes listar los archivos y directorios dentro de un directorio utilizando el método listFiles(), como muestra la imagen:

```java
import java.io.File;

public class Main {
    public static void main(String[] args) {
        File directorio = new File("mi_directorio");
        File[] archivos = directorio.listFiles();
        if (archivos != null) {
            for (File archivo : archivos) {
                System.out.println(archivo.getName());
            }
        }
    }
}
```

Fig.14. Listado de ficheros de un directorio.

13.3. Eliminar un directorio

Puedes eliminar un directorio utilizando el método delete() de la clase File, como muestra la imagen:

```java
import java.io.File;

public class Main {
    public static void main(String[] args) {
        File directorio = new File("mi_directorio");
        if (directorio.exists()) {
            if (directorio.delete()) {
                System.out.println("Directorio eliminado correctamente.");
            } else {
                System.out.println("Error al eliminar el directorio.");
            }
        }
    }
}
```

Fig.15. Eliminación de un directorio.

/ 14. Creación y eliminación de archivos

En Java, se pueden crear y eliminar archivos de dos maneras:

- **Mediante el uso de la clase File**, para las operaciones básicas.
- **Mediante el uso de las clases Path, Files y FileOutputStream**, para operaciones más avanzadas.

Vamos a ver un ejemplo de cada uno de estos usos.

- **Uso de la clase File:**

La clase File proporciona **métodos básicos para manipular archivos y directorios**, como crear, eliminar, renombrar y comprobar propiedades.

```java
import java.io.File;
import java.io.IOException;

public class Main {
    public static void main(String[] args) {
        // Crear un archivo
        File archivo = new File("archivo.txt");
        try {
            if (archivo.createNewFile()) {
                System.out.println("Archivo creado: " + archivo.getName());
            } else {
                System.out.println("El archivo ya existe.");
            }
        } catch (IOException e) {
            System.out.println("Error al crear el archivo.");
            e.printStackTrace();
        }

        // Eliminar un archivo
        if (archivo.delete()) {
            System.out.println("Archivo eliminado: " + archivo.getName());
        } else {
            System.out.println("No se pudo eliminar el archivo.");
        }
    }
}
```

Fig.16. Uso de la clase File.

- **Uso de las clases Path, Files y FileOutputStream**

La clase Path proporciona **métodos más avanzados y flexibles para manejar archivos y directorios**.

```java
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;

public class Main {
    public static void main(String[] args) {
        // Crear un archivo
        Path archivoPath = Paths.get("archivo.txt");
        try {
            Files.createFile(archivoPath);
            System.out.println("Archivo creado: " + archivoPath.getFileName());
        } catch (IOException e) {
            System.out.println("Error al crear el archivo.");
            e.printStackTrace();
        }

        // Eliminar un archivo
        try {
            Files.deleteIfExists(archivoPath);
            System.out.println("Archivo eliminado: " + archivoPath.getFileName());
        } catch (IOException e) {
            System.out.println("No se pudo eliminar el archivo.");
            e.printStackTrace();
        }
    }
}
```

Fig.17. Uso de las clases Path, File y FileOutputStream.

Ambos ejemplos logran lo mismo: crean un archivo llamado **'archivo.txt'** en el directorio actual y luego lo eliminan. Sin embargo, a diferencia de la primera opción (primera imagen), la segunda (segunda imagen) utiliza las nuevas API de manejo de archivos introducidas en Java 7, que proporcionan un manejo más robusto y flexible de las operaciones de archivo.

/ 15. Resumen y resolución del caso práctico de la unidad

A lo largo de este tema, hemos visto que podemos **manejar**, de una forma relativamente simple, los fallos en nuestros programas o, como se conoce a estos, las **excepciones**, mediante el bloque try-catch.

También hemos aprendido a **propagar una excepción** ocurrida de forma correcta, así como a **crear nuestras propias excepciones** de una manera muy sencilla, pudiéndolas utilizar como una excepción más.

A continuación se ha visto cómo **realizar operaciones de escritura y lectura con ficheros de texto** y cómo podemos **tratar directorios desde Java**, pudiendo crearlos, listarlos y eliminarlos.

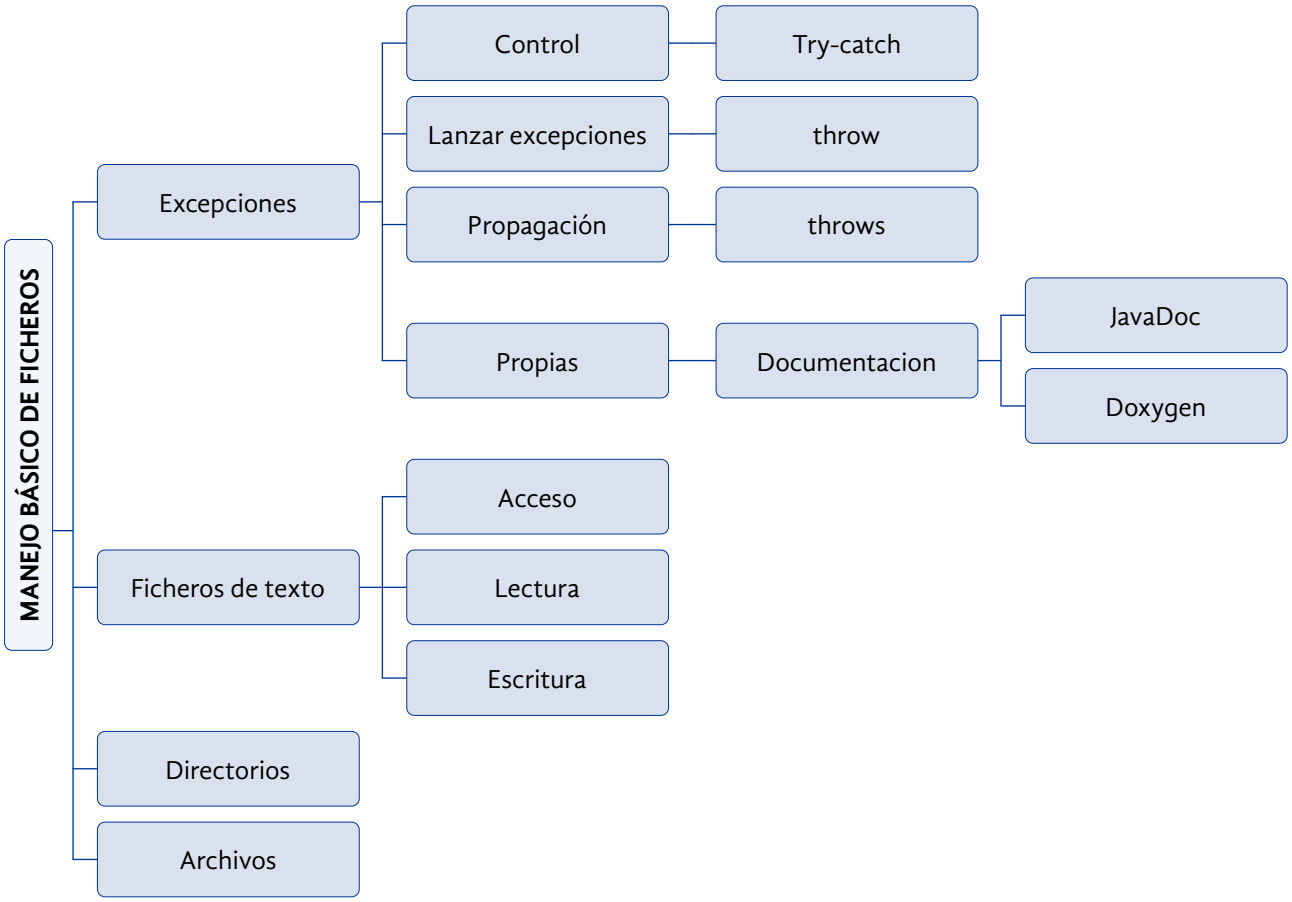


Fig.18. Esquema resumen del tema .

Resolución del caso práctico de la unidad

Cuando los programas empiezan a ser más complicados, es normal que aparezcan errores de código que no sabemos de dónde vienen.

Las excepciones nos van a ayudar en esto, ya que, según la excepción que se produzca, podremos saber qué tipo de error ha ocurrido y, sumando a esto que NetBeans nos da una estimación de dónde ha ocurrido el error, nos será un poco más sencilla su localización.

Con el tratamiento de excepciones vamos a conseguir que, cuando una de ellas se produzca, podamos tratarla y hacer que el programa no se cierre inesperadamente, pudiendo volver a su ejecución normal.

/ 16. Bibliografía

Bloch, J. (2018). *Effective Java* (3.ª ed.). Addison-Wesley.

Caro, C. M., Ramos, A. N. y Barceló, A. V. (2002). *Introducción a la programación con orientación a objetos*. Prentice Hall.

Flujos de datos (s. f.). Recuperado de: http://www.sc.ehu.es/sbweb/fisica/cursoJava/fundamentos/archivos/flujos.htm

Knuth, D. E. (1997). *The Art of Computer Programming (Vol. 1: Fundamental Algorithms)* (3.ª ed.). Addison-Wesley.

Manuel, P. G. J. (2022). *Entornos de desarrollo*. Ediciones Paraninfo, S.A.

Ojeda, F. C. (2001). *Introducción a la programación*. Anaya Multimedia.

Schildt, H. (2018). *Java: The complete reference* (11.ª ed.). McGraw-Hill Education.